System Detective

CIE-1, Computer Organisation & Operating Systems (CO1) | TE IT | Roll No. __ , Name _____ |
Submitted 2 September 2026

1. The machine under inspection

Every number in this report came from one machine, an Ubuntu 24.04 guest running under a KVM hypervisor. I am stating the virtualisation up front because it shapes several results below: the guest is given a single logical CPU, no swap is configured, and the cache sizes reported are the host's, not a desktop's. A reader who assumed a normal laptop would find some of my numbers strange. All five commands can be re-run live at the viva and I have kept the raw text output alongside the screenshots.

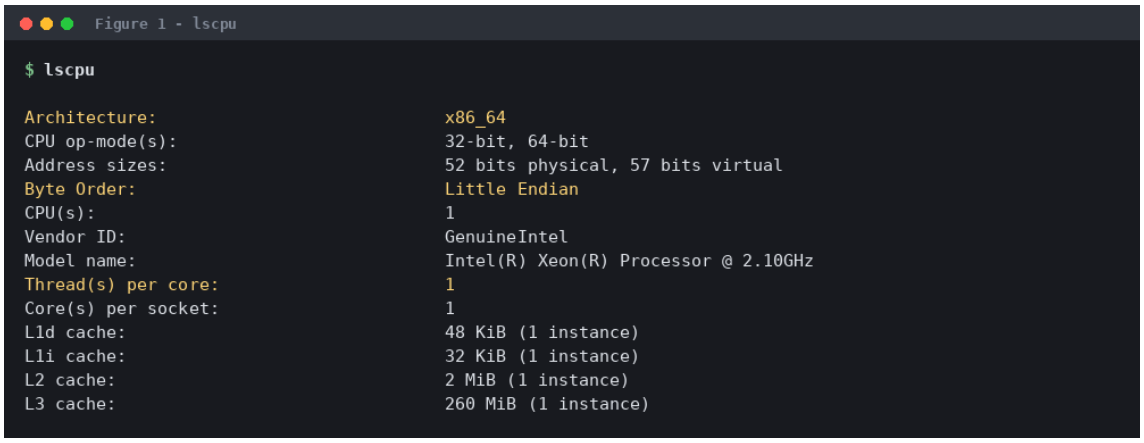


Figure 1. lscpu, filtered to the architecture and cache fields. Highlighted: 64-bit x86_64 ISA, little-endian byte order, one thread per core.

2. Instruction set and what it tells me

The ISA here is x86_64: a 64-bit, little-endian, CISC instruction set with variable-length instructions. Little-endian means the least significant byte of a multi-byte value sits at the lowest address, which is why a four-byte integer 1 appears in memory as 01 00 00 00 rather than 00 00 00 01. The address sizes line reports 52 physical and 57 virtual bits, so the virtual address space the OS can hand out is far larger than the physical memory actually installed. That gap is the whole reason paging and virtual memory exist.

The flags field lists 114 ISA extensions. Several of them, including avx2 and avx512f, are SIMD extensions that operate on 256 or 512 bits at a time. This matters for the experiment in section 5: a compiler that vectorises a loop can only do so usefully when the data it needs is contiguous, so the ISA and the memory layout are not independent choices.

The cache hierarchy is 48 KiB of L1 data cache, 2 MiB of L2 and 260 MiB of L3. L1 is roughly a thousandth the size of the working set I use later, which is the point: caches only help if the access pattern lets them.

3. Memory at the moment of capture

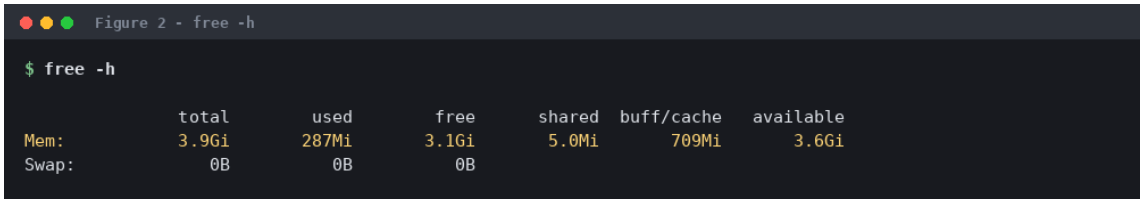


Figure 2. free -h. Highlighted row: total 3.9 GiB, used 287 MiB, free 3.1 GiB, buff/cache 709 MiB, available 3.6 GiB. Swap is 0 B.

The column that confused me first was the difference between free and available. Free counts pages the kernel is holding untouched. Available is free plus the part of buff/cache the kernel could reclaim if a process asked for memory. On this capture that is a 500 MiB difference. The page cache is not wasted memory; it is the kernel keeping recently read file blocks around because reading them again from disk would be far slower. A system showing very little free memory and plenty of available memory is behaving correctly, not running out.

Swap is 0 B on this machine, so there is no backing store to page out to. If allocations exceed physical memory the kernel will not slow down gracefully; the OOM killer will terminate a process instead. This is the single most important thing the AI got wrong for me, and it is documented in ai_log.pdf.

4. Processes, threads and states

Figure 3 - ps -eLf (top by CPU)

```
$ ps -eLf --sort=-pcpu | head

UID        PID  PPID   LWP  C NLWP STIME TTY          TIME CMD
root         1     0     1  1    8 09:31 ?        00:00:00 /process_api --firecracker-init --addr 0.0.0.0:202
root       435     1   435  1   22 09:31 ?        00:00:00 /opt/rclone/rclone-filestore multimount --config /
root        11     2    11  0    1 09:31 ?        00:00:00 [kworker/0:1-virtio_vsock]
root        12     2    12  0    1 09:31 ?        00:00:00 [kworker/u4:0-events_unbound]
root        15     2    15  0    1 09:31 ?        00:00:00 [rcu_preempt]
root       398     2   398  0    1 09:31 ?        00:00:00 [kworker/u4:5-events_unbound]
root        80     2    80  0    1 09:31 ?        00:00:00 [kworker/u4:2-events_unbound]
root        14     2    14  0    1 09:31 ?        00:00:00 [ksoftirqd/0]
root         2     0     2  0    1 09:31 ?        00:00:00 [kthreadd]
root         3     2     3  0    1 09:31 ?        00:00:00 [pool_workqueue_release]
```

Figure 3. ps -eLf sorted by CPU. Note PID 1 repeating across several rows with different LWP values: one process, eight kernel-scheduled threads.

ps -eLf prints one row per thread rather than per process, which makes the distinction visible. PID 1 appears eight times with the same PID and different LWP values and NLWP of 8. Those eight threads share one address space, one set of open file descriptors and one page table, but each is scheduled independently. That is exactly the trade the thread abstraction makes: cheaper context switching and shared memory, at the cost of having to synchronise access to that shared memory yourself.

The STAT letters on my dashboard are the scheduler states. S is interruptible sleep, meaning the thread is blocked waiting for an event such as I/O and can be woken by a signal. Nearly everything on an idle machine sits in S, which surprised me until I thought about it: a process is only in R when it is actually on a CPU or ready to be. I is an idle kernel thread. D would be uninterruptible sleep, normally a thread part-way through a disk operation that cannot safely be interrupted, and a machine with many D-state processes is usually I/O bound rather than CPU bound.

Figure 4 - strace -c

```
$ strace -c ls /usr/bin

% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         7           read
  0.00    0.000000           0         3           write
  0.00    0.000000           0         9           close
  0.00    0.000000           0         8           fstat
  0.00    0.000000           0        18           mmap
  0.00    0.000000           0         5           mprotect
  0.00    0.000000           0         1           munmap
  0.00    0.000000           0         5           brk
  0.00    0.000000           0         2         2 ioctl
  0.00    0.000000           0         2           pread64
  0.00    0.000000           0         2         2 access
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         2         1 statfs
  0.00    0.000000           0         1           arch_prctl
  0.00    0.000000           0         3           getdents64
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         7           openat
  0.00    0.000000           0         1           set_robust_list
  0.00    0.000000           0         1           prlimit64
  0.00    0.000000           0         1           getrandom
  0.00    0.000000           0         1           statx
  0.00    0.000000           0         1           rseq
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        82         5 total
```

Figure 4. strace -c on ls. 82 system calls in total, 5 returning errors. Highlighted: 18 mmap calls and 7 openat calls.

strace -c counts the boundary crossings between user mode and kernel mode. Listing one directory took 82 system calls. The 18 mmap calls are mostly the dynamic linker mapping shared libraries into the address space before main() ever runs, and the 3 getdents64 calls are the actual directory read. The 5 errors are not failures of the program: two are access checks and two are ioctl calls probing whether stdout is a terminal, which is how ls decides whether to colourise. Seeing this made the user/kernel boundary concrete for me. Every one of these rows is a mode switch that a purely user-space instruction would not need.

5. Locality of reference: the experiment

I allocated an 8192 x 8192 array of doubles, 512 MiB, and summed every element twice. The first traversal walks along rows, which in C matches the row-major storage order, so consecutive iterations touch consecutive addresses. The second walks down columns, so each step jumps 8192 x 8 = 65,536 bytes. The arithmetic is identical: the same 67.1 million additions in both cases. Only the order of memory access differs. I touched every page before timing so that page-fault cost was not counted, compiled with gcc -O2, and ran three iterations.

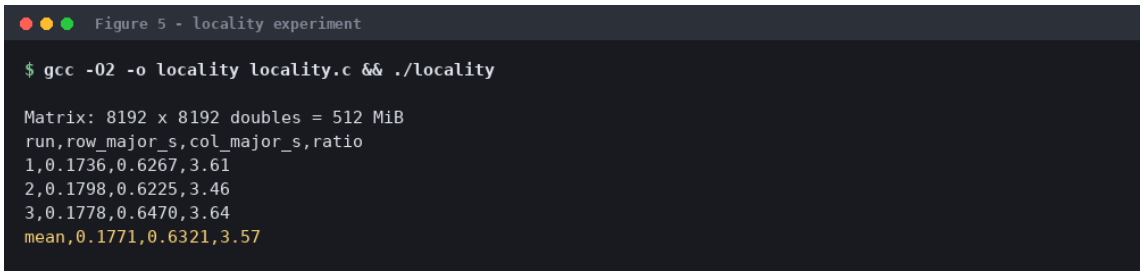


Figure 5. Three runs. Row-major mean 0.1771 s, column-major mean 0.6321 s, ratio 3.57x. The spread across runs is under 4%.

Column-major traversal is 3.57 times slower on this machine, consistently across three runs. The cause is the cache line. Memory moves between RAM and cache in 64-byte lines, and a double is 8 bytes, so one line holds 8 array elements. Walking along a row, one cache miss is amortised over the next 8 accesses. Walking down a column, each access lands in a different line, so the line fetched is used once and evicted long before the loop returns to it. The same instruction stream, the same data, and a 3.5x difference produced entirely by the order of reference.

I expected a larger gap. A naive reading of the 8-elements-per-line argument predicts roughly 8x, and that is what the AI predicted too. The measured 3.57x is lower, and I think the reason is hardware prefetching: the column walk has a large but perfectly regular stride, and the prefetcher can partly follow it, so some lines arrive before they are demanded. The theoretical ratio assumes every miss costs full latency, which is not true on a modern core. This gap between the textbook prediction and my measurement is discussed in the AI log.

6. The dashboard

dashboard/dashboard.py is a single-file Python program using only the standard library, so it needs no pip install to demonstrate. It serves one page on localhost:8000 and re-reads lscpu, /proc/meminfo and ps on every request; the page refreshes itself every five seconds. Nothing is cached and nothing is hardcoded. The only value read from a file is the locality timing, because that experiment takes about three seconds and is measured separately by the C program.



Figure 6. The dashboard as served. The clock in the header advances on each refresh, and the process table reorders as load changes. In this capture the screenshot tool itself is the top process at 75% CPU, which is a useful reminder that observing a system perturbs it.

7. Honesty and reproduction

No value in this report was typed by hand. To reproduce: run the five commands, then gcc -O2 -o locality locality.c && ./locality > outputs/06_locality.txt, then python3 dashboard/dashboard.py and open localhost:8000. The memory and process figures will differ from mine, because they describe a machine at a moment. The locality ratio should stay in the same range on similar hardware, though a machine with a smaller L3 or a different prefetcher will not give exactly 3.57.